

Esame OS Internals (Cabodi)

Data: 26/10/2022

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Contesto: Sia dato il frammento di programma rappresentato:

```
#define N 128
#define M 64
int i, j, k;
float *A[N], B[N][M];

for(i=0; i<N; i++) {
    A[i] = calloc(M * sizeof(float));
    for (j=0; j<M; j++) {
        B[i][j] = i * j;
    }
}

for(k=0; k<N*M; k++) {
    i = rand() % N;
    j = rand() % M;
    A[i][j] += B[i][j];
}
```

Il codice macchina generato da tali istruzioni viene eseguito in un sistema con memoria virtuale gestita mediante paginazione, con pagine di 4 Kbyte, utilizzando come politica di sostituzione pagine la LRU (Least Recently Used). Si sa che i puntatori occupano 64 bit, mentre sia `int` che `float` occupano 32 bit. Si sa inoltre che l'indirizzo logico (virtuale) ha una dimensione di 32 bit, mentre l'indirizzo fisico occupa 24 bit.

Si risponda alle seguenti domande:

- Si calcoli la dimensione massima possibile (in Byte e in numero di pagine/frame) del Virtual Address Space (VAS) di un processo, e la dimensione massima della RAM.
- Si calcoli la dimensione in memoria (espressa in numero di pagine) del codice e dei dati, necessari all'esecuzione del frammento proposto. In particolare si contino le pagine necessarie per il codice (assumere per il codice una dimensione fissa di 1 KByte), le pagine per lo stack, le pagine necessarie a contenere A (solo l'array di puntatori) e le pagine necessarie a contenere B. Se si fanno ipotesi su come sia allocata la memoria (sia stack che data/heap), dichiarare e motivare tali ipotesi.
- Si supponga ora che, durante l'esecuzione del secondo ciclo (`for k...`), il resident set per il processo in esecuzione sia fisso e limitato a F frames. Supponendo di disporre dei 3

frame sufficienti a contenere il codice, lo stack ed **A**, quanti frame supplementari occorre assegnare al processo affinché, per accessi casuali alla matrice **B**, e per un numero elevato di iterazioni, il numero dei page fault sia minimizzato? Se se ne assegnano meno, quale è il tasso di page fault atteso (page fault / memoria memory reference)?

Domanda 2 (I/O e Chiamate di Sistema)

Contesto: Si consideri il sottosistema di I/O (Input/Output). Un processo richiede una operazione di I/O mediante la System Call `read(fd, buffer, N)`. Si supponga che il dispositivo dal quale leggere i dati disponga di un controller con supporto DMA (Direct Memory Access).

Si risponda alle seguenti domande:

- Che cosa indicano `buffer` ed `N`? Il `buffer` indicato dal chiamante (nella memoria user) viene utilizzato per il DMA (il dispositivo trasferisce direttamente in esso i dati)? O si utilizza un buffer nel kernel? Motivare.
 - Il processo, dopo la chiamata, attende i dati. L'attesa viene realizzata mediante attesa attiva (polling) o interruzioni (interrupt)? Motivare, indicando il ruolo del sistema operativo e della CPU.
 - È possibile che l'operazione `read` sia completata per un numero di Byte inferiore a `N`? Motivare, proponendo almeno un caso/situazione a titolo di esempio.
-

Domanda 3 (Allocazione File System e Prestazioni)

Contesto: Si considerino tre possibili meccanismi per l'allocazione dei blocchi di un File System: Contigua, FAT (File Allocation Table), basata su Inode (organizzato con indici diretti, e uno o più livelli di indirizzamento indiretto).

Si risponda alle seguenti domande (sempre confrontando le tre strategie):

- Da quale tipo di frammentazione (interna o esterna) sono penalizzate le 3 strategie? E quali subiscono degrado di prestazioni causa "file system aging" (invecchiamento del FS a fronte di cicli ripetuti di creazione/cancellazione)?
- Si supponga di voler leggere, all'interno del file **F**, il blocco **B** (ad esempio il blocco 1000 del file), e che i dati che consentono di localizzare il file (nella directory) siano già stati letti in memoria, assieme a eventuali blocchi speciali per il file (es. FAT o Inode). Quale è il tempo necessario ad accedere a **B**? Si dia una stima del numero di letture di blocchi disco per localizzare e leggere **B**.
- La dimensione massima possibile per un file dipende dalla strategia? (motivare).

SECTION 2: OS161

Domanda 4 (Traduzione Indirizzi e dumbvm)

Contesto: Viene fornito un sistema OS161, in esecuzione su un simulatore MIPS `sys161` con 8 MB di memoria RAM. È noto che il processo P1 ha uno spazio di indirizzamento (nella realizzazione proposta dalla `dumbvm`) con le variabili `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2` aventi i seguenti valori: `0x200000`, `0x100000`, `0x4000`, `0x8000`, 3, 4. Lo stack ha dimensione fissa (`DUMBVM_STACKPAGES` pari a 18), ed è allocato all'indirizzo fisico `0x300000`. La dimensione di pagina nel MIPS è di 4KB.

Si risponda alle seguenti domande (si forniscano espressioni o dettagli necessari al calcolo e non unicamente il risultato finale. I risultati esadecimali vanno forniti con il prefisso `0x`):

- Tradurre i seguenti indirizzi logici utente, o dire se causano errore (segmentation fault):
 1. `0x3010`
 2. `0x5010`
 3. `0x8030`
 4. `0x9020`
- Per ognuno dei seguenti indirizzi fisici, dire se fanno parte dello spazio di indirizzamento utente. In caso affermativo, fornire il corrispondente indirizzo logico utente e il corrispondente indirizzo logico kernel.
 1. `0x401050`
 2. `0x800100`

Domanda 5 (Gestione Processi e PIDs)

Contesto: Si consideri, in OS161, la realizzazione del supporto ai processi. Si risponda alle seguenti domande:

- Perché il processo che termina (con la system call `sys_exit`) NON effettua una chiamata a `proc_destroy(curproc)`?
- Si vogliono realizzare due funzioni per convertire fra il puntatore a un processo e il relativo PID (il cui risultato viene ottenuto "by pointer", mentre il valore di ritorno è un codice di errore, 0 se operazione conclusa con successo):

```
int proc_to_pid(struct proc *p, pid_t *ppid);
int proc_from_pid(pid_t pid, struct proc **pproc);
```

Quesito: Si realizzino le due funzioni. NON è necessario scrivere codice C se non per definire strutture dati o altre funzioni. Ma è necessario spiegare a parole cosa siano (e come andrebbero inizializzate) eventuali strutture dati necessarie per effettuare le due conversioni.

ANSWERS

Domanda 1 (Memoria Virtuale, Paginazione e Codice C)

Dati Iniziali:

- $N = 128$ (2^7); $M = 64$ (2^6)
- Dimensione 'float' e 'int' = 32 bit = 4 Byte (2^2 Byte)
- Dimensione puntatore = 64 bit = 8 Byte (2^3 Byte)
- Dimensione Pagina = 4 KB = 4096 Byte (2^{12} Byte)

A) Dimensione massima del VAS e della RAM:

L'architettura presenta indirizzi logici a 32 bit e indirizzi fisici a 24 bit.

- **Virtual Address Space (VAS):**
 - In Byte: 2^{32} Byte = **4 GB**.
 - In numero di pagine: $\frac{2^{32}}{2^{12}} = 2^{20} = \mathbf{1.048.576}$ pagine (1 Mega-pagina).
- **RAM massima:**
 - In Byte: 2^{24} Byte = **16 MB**.
 - In numero di frame: $\frac{2^{24}}{2^{12}} = 2^{12} = \mathbf{4096}$ frame.

B) Dimensione in memoria (numero di pagine) di Codice e Dati:

- **Codice:** Da traccia, assumiamo una dimensione fissa di 1 KB. Sarà contenuta in **1 pagina**.
- **Array A (solo puntatori):** L'array contiene $N = 128$ puntatori da 8 Byte.

$$\text{Dimensione } A = 128 \times 8 \text{ B} = 1024 \text{ B.}$$

Tale dimensione è ampiamente inferiore a 4096 Byte, pertanto richiede **1 pagina**.

- **Matrice B:** La matrice contiene $N \times M$ elementi di tipo 'float' (4 Byte).

$$\text{Dimensione } B = 128 \times 64 \times 4 \text{ B} = 2^7 \times 2^6 \times 2^2 = 2^{15} \text{ B} = 32.768 \text{ Byte} = 32 \text{ KB.}$$

In numero di pagine: $\frac{32 \text{ KB}}{4 \text{ KB}} = \mathbf{8}$ pagine.

- **Stack:** Assumendo che gli array A e B siano allocati globalmente (nel segmento dati/.bss), lo stack dovrà ospitare solamente le variabili intere locali i, j, k ($3 \times 4 = 12$ Byte) e i record di attivazione delle funzioni. Richiederà quindi l'allocazione base di **1 pagina**.

C) Resident Set e Tasso di Page Fault:

- **Frame Supplementari necessari:** Durante il secondo ciclo, l'algoritmo accede alla matrice B in posizioni puramente casuali (tramite `rand()`). Non essendoci località spaziale prevedibile, l'algoritmo LRU non offre alcun vantaggio. L'unico modo per minimizzare (azzerare) i page fault per la matrice B è mantenerla interamente all'interno del Resident Set. Essendo B grande 8 pagine, occorre assegnare al processo **8 frame supplementari**.
- **Tasso di Page Fault atteso ($n < 8$):** Supponendo di disporre di n frame per la matrice B (con $0 < n < 8$), e assumendo una distribuzione pseudo-casuale uniforme degli accessi generati dalla funzione `rand()`, la probabilità che l'elemento richiesto si trovi in una delle n pagine residenti in RAM (Hit Rate) è pari a $\frac{n}{8}$. Di conseguenza, il tasso di page fault atteso (ovvero il Miss Rate per ogni memory reference a B) corrisponde alla probabilità complementare:

$$\text{Page Fault Rate} = 1 - \frac{n}{8} = \frac{8 - n}{8}$$

Domanda 2 (I/O e Chiamate di Sistema)

- **A) Significato di buffer ed N - Buffer Utente vs Kernel:**

I parametri indicano rispettivamente il puntatore all'area di memoria utente in cui depositare i dati (`buffer`) e la quantità di byte che il processo richiede di leggere (N).

Utilizzo nel DMA: Il trasferimento DMA **non avviene direttamente nel buffer utente**. Il controller trasferisce i dati in un **buffer nel kernel** pre-allocato, e solo successivamente la CPU copia i dati da quest'ultimo al buffer utente.

Motivazione: Il DMA opera in background utilizzando indirizzi fisici. Il buffer utente (allocato con indirizzi virtuali) potrebbe subire un *page-out* (sfratto su disco) da parte del SO durante l'I/O asincrono. Se il DMA scrivesse direttamente su quel frame di RAM ora riassegnato, causerebbe grave corruzione della memoria. Inoltre, un'operazione di lettura parziale (es. $N = 50$ byte) da un disco richiede comunque la lettura di un intero blocco fisico (es. 4 KB), imponendo al kernel di usare un buffer di staging per la ricezione dell'intero blocco.

- **B) Gestione dell'attesa (Polling vs Interrupts):**

L'attesa viene gestita **esclusivamente tramite interruzioni (interrupts)**.

Motivazione: I dispositivi meccanici e di rete sono svariati ordini di grandezza **più lenti** rispetto alle moderne CPU (es. latenze di millisecondi per il disco contro nanosecondi per la CPU). Un'attesa attiva (polling) terrebbe occupata la CPU in un ciclo vuoto controllando il registro di stato del DMA, sprecando milioni di cicli utili.

Ruolo del SO e CPU: All'invocazione della `read`, il SO programma il DMA e immediatamente sospende il processo corrente (mettendolo in stato di *Wait/Blocked*), avviando il Context Switch verso un altro processo (*Preemption* volontaria). Quando il DMA termina di riempire il buffer nel kernel, alza una linea di *Interrupt* hardware. La CPU interrompe temporaneamente il processo in esecuzione corrente per eseguire la Interrupt Service Routine (ISR) del SO, la quale sposta il processo originale dalla coda di attesa alla *Ready Queue*.

- **C) Possibilità di una lettura incompleta ($< N$ Byte):**

SI, è assolutamente possibile.

Motivazione ed Esempi: La system call `read()` garantisce solo di restituire *fino a* N byte. Ritorna il numero di byte effettivamente letti, oppure 0 o codici negativi in caso di errore.

Esempi:

1. **End of File (EOF):** Se il file su disco contiene in totale 30 byte dalla posizione corrente e la richiesta **read** è per $N = 100$, il sistema trasferirà solo i 30 byte disponibili, arrestandosi all'EOF e restituendo 30.
2. **Dispositivi a carattere (Stream / Socket):** Se si esegue una **read** bloccante su un socket TCP o su una *pipe* per $N = 4096$ byte, ma nel buffer di ricezione del kernel sono arrivati finora solo pacchetti per 500 byte, la chiamata si sbloccherà immediatamente ritornando 500 per evitare di bloccare il processo nell'attesa incerta dei restanti dati dalla rete.

Domanda 3 (Allocazione File System e Prestazioni)

A) Frammentazione e File System Aging:

- **Frammentazione Interna:** Tutte e tre le strategie (Contigua, FAT, Inode) soffrono dello stesso grado di frammentazione interna. Essa si verifica nell'ultimo blocco allocato al file, sprecando mediamente mezza pagina/blocco per ogni file.
- **Frammentazione Esterna:** Solo l'**Allocazione Contigua** ne è penalizzata. Richiedendo blocchi fisicamente consecutivi, lascia "buchi" di spazio inutilizzabile troppo piccoli per ospitare nuovi file. FAT e Inode, allocando dinamicamente blocchi sparsi tramite puntatori o indici, sono immuni da questo problema.
- **File System Aging:** L'**Allocazione Contigua** subisce un gravissimo degrado a causa dell'invecchiamento. Cicli ripetuti di creazione e cancellazione frantumano lo spazio libero (frammentazione esterna progressiva), richiedendo costose operazioni di deframmentazione (compattazione) per poter allocare file di grandi dimensioni. FAT e Inode non subiscono degrado in termini di *allocabilità*, ma possono subire un degrado *prestazionale* (i blocchi di un file diventano molto sparpagliati, aumentando il tempo di *seek* della testina del disco).

B) Accesso al blocco logico B (es. 1000) con metadati in RAM:

- **Allocazione Contigua:** Nell'allocazione contigua *non ci sono puntatori* tra i blocchi. Il file è un array fisico sul disco. Noto l'indirizzo di base (dal FCB in RAM), l'indirizzo fisico si calcola matematicamente: $\text{Indirizzo Fisico} = \text{Base} + 1000$.
Costo: **0 letture in RAM**, **1 singola lettura su disco**.
- **Allocazione FAT:** La FAT è una lista concatenata di indici residente in RAM. Per trovare il blocco 1000, il sistema operativo deve scorrere la tabella partendo dal primo indice del file (salvato nel FCB) e seguire i puntatori in RAM per 1000 volte.
Costo: **1000 letture in RAM** (rapide), seguite da **1 singola lettura su disco** all'indirizzo fisico trovato.
- **Allocazione Inode:** L'Inode è una struttura ad albero indicizzata. Trovandosi l'Inode interamente in RAM (inclusi eventuali blocchi indiretti menzionati nella traccia), il sistema calcola matematicamente quale puntatore (diretto o indiretto) contiene l'indirizzo del blocco 1000, e vi accede direttamente come in un array.
Costo: **Poche operazioni matematiche in RAM** (1 o 2 accessi agli array dell'Inode), **1 singola lettura su disco**.

C) Dipendenza della dimensione massima del file dalla strategia:
SI, la dimensione massima dipende fortemente dalla strategia di allocazione.

- **Allocazione Contigua:** La dimensione massima non dipende dalla grandezza del disco, ma dalla dimensione del **più grande buco di spazio contiguo libero** disponibile in quel preciso istante.
- **Allocazione FAT:** La dimensione massima è limitata esclusivamente dalla dimensione totale del file system (meno lo spazio per la FAT stessa). Un file può teoricamente occupare tutti i blocchi liberi del disco, anche se frammentati.
- **Allocazione Inode:** La dimensione massima ha un **limite architetturale intrinseco**. Dipende dal numero di puntatori inseriti nella struttura dell'Inode (es. 12 diretti, 1 indiretto singolo, 1 doppio, 1 triplo). Indipendentemente da quanto spazio libero vi sia sul disco, il file non può superare la dimensione massima che l'albero dei puntatori dell'Inode è in grado di indirizzare (es. limite storico di 4 GB su filesystem Ext2/Ext3 a 32 bit).

Domanda 4 (Traduzione Indirizzi e dumbvm)

A) Traduzione indirizzi logici utente:

Le dimensioni dei segmenti sono determinate dal numero di pagine allocate (4 KB = 0x1000 Byte per pagina).

- **Segmento 1:** Base logica 0x4000, dimensione 3 pagine (0x3000). Range logico: [0x4000, 0x6FFF]. Base fisica: 0x200000.
 - **Segmento 2:** Base logica 0x8000, dimensione 4 pagine (0x4000). Range logico: [0x8000, 0xBFFF]. Base fisica: 0x100000.
1. **0x3010:** L'indirizzo è inferiore alla base del segmento 1 (0x4000). Non appartiene ad alcuno spazio valido mappato dal processo. Genera un **Segmentation Fault** (Non Page Fault, in quanto dumbvm è un sistema senza paginazione a richiesta o swap).
 2. **0x5010:** Appartiene al Segmento 1.
 $\text{Offset} = 0x5010 - 0x4000 = 0x1010$.
 $\text{Indirizzo Fisico} = 0x200000 + 0x1010 = \mathbf{0x201010}$.
 3. **0x8030:** Appartiene al Segmento 2.
 $\text{Offset} = 0x8030 - 0x8000 = 0x0030$.
 $\text{Indirizzo Fisico} = 0x100000 + 0x0030 = \mathbf{0x100030}$.
 4. **0x9020:** Appartiene al Segmento 2.
 $\text{Offset} = 0x9020 - 0x8000 = 0x1020$.
 $\text{Indirizzo Fisico} = 0x100000 + 0x1020 = \mathbf{0x101020}$.

B) Verifica appartenenza indirizzi fisici allo spazio utente:

Per stabilire se un indirizzo fisico appartenga al processo in esame, occorre calcolare i **range fisici** effettivamente allocati per i suoi segmenti:

- **Fisico Seg 1:** Da 0x200000 a $0x200000 + 0x3000 - 1 = \mathbf{0x202FFF}$.
- **Fisico Seg 2:** Da 0x100000 a $0x100000 + 0x4000 - 1 = \mathbf{0x103FFF}$.
- **Fisico Stack:** Lo stack occupa 18 pagine ($18 \times 0x1000 = 0x12000$ Byte) ed ha base fisica a 0x300000.
 $\text{Range Fisico: Da } 0x300000 \text{ a } 0x300000 + 0x12000 - 1 = \mathbf{0x311FFF}$.

1. **0x401050 - Appartiene all'utente? NO.**

Motivazione: L'indirizzo fisico 0x401050 è strettamente maggiore dell'ultimo indirizzo fisico utile per questo processo (che è la fine dello stack a 0x311FFF). Pertanto non appartiene allo spazio utente di questo processo (anche se la sua traduzione logica kernel kseg0 sarebbe 0x80401050).

2. **0x800100 - Appartiene all'utente? NO.**

Motivazione: Non solo non appartiene ai segmenti del processo, ma questo indirizzo è **fuori dai limiti della memoria fisica hardware**. La traccia specifica che la RAM totale è di 8 MB. Poiché $8 \text{ MB} = 8 \times 2^{20} \text{ Byte} = 0x800000 \text{ Byte}$, il limite superiore dell'hardware è 0x7FFFFFFF. L'indirizzo 0x800100 è un indirizzo inesistente nel simulatore.

Domanda 5 (Gestione Processi e PIDs)

A) Perché `sys_exit` non effettua una chiamata a `proc_destroy(curproc)`?

I motivi principali sono due, legati alla sicurezza della memoria e alla logica di sincronizzazione di UNIX:

1. **Use-After-Free (Impossibilità di "suicidio"):** La chiamata a `sys_exit` viene eseguita in kernel mode dal thread appartenente al processo corrente. Chiamare `proc_destroy(curproc)` equivarrebbe a distruggere (liberando tramite `kfree`) l'area di memoria e le strutture (incluso il VFS e lo spazio di indirizzamento) che il thread stesso sta attualmente utilizzando per poter terminare la sua esecuzione. Questo provocherebbe istantaneamente un Kernel Panic. Il processo ha bisogno di un ultimo *context switch* prima che le sue strutture possano essere distrutte in sicurezza.
2. **Stato Zombie e `waitpid()`:** Il processo che termina deve conservare il proprio codice di uscita (*exit status*) affinché il processo padre possa leggerlo chiamando `waitpid()`. Il processo diventa uno "Zombie". Spetterà al processo padre (tramite la sua `waitpid`), o a un processo mietitore, effettuare la reale `proc_destroy` per pulire la Process Table dopo aver prelevato l'*exit status*.

B) Strutture e funzioni per la conversione Processo $\leftrightarrow$ PID:

Strutture dati necessarie e inizializzazione: Per effettuare le conversioni è necessario definire una **Process Table** (Tabella dei Processi) a livello globale nel kernel.

- **Struttura:** Un array globale di puntatori a processo: `struct proc *ptable[PID_MAX]`; (dove l'indice dell'array corrisponde implicitamente al PID assegnato).
- **Sincronizzazione:** Un Lock o Spinlock dedicato (es. `struct lock *ptable_lock;`) per proteggere l'array globale da accessi concorrenti durante chiamate parallele.
- **Inizializzazione:** Durante il boot, la tabella deve essere interamente azzerata, impostando ogni puntatore a NULL, e il lock deve essere creato.

Realizzazione Logica delle funzioni:

1. `int proc_from_pid(pid_t pid, struct proc **pproc);`

Questa funzione riceve un `pid` numerico in ingresso e restituisce il relativo puntatore.

Logica:

1. Acquisire il Lock della Process Table.
2. Effettuare un *bounds checking*: verificare che `pid > 0` e `pid < PID_MAX`. Se falso, rilasciare il lock e ritornare un codice di errore (es. `ESRCH`).

3. Verificare se l'elemento in `ptable[pid]` è NULL (PID non assegnato/terminato). Se sì, ritornare errore.
 4. Se valido, dereferenziare il risultato: `*pproc = ptable[pid];`
 5. Rilasciare il lock e ritornare 0 (successo).
2. `int proc_to_pid(struct proc *p, pid_t *ppid);`
 Questa funzione cerca il processo fornito all'interno della tabella e ne restituisce l'indice (PID).
Logica:
1. Acquisire il Lock della Process Table.
 2. Verificare che il puntatore `p` in ingresso non sia NULL. (Se lo è, ritornare EINVAL).
 3. Eseguire un ciclo `for` scansionando l'intero array `ptable` da $i = 1$ a `PID_MAX-1`.
 4. Ad ogni iterazione, verificare se `ptable[i] == p`. Se viene trovata un'uguaglianza, impostare `*ppid = i;`, rilasciare il lock e ritornare 0.
 5. Se il ciclo termina senza successi, significa che il processo non è registrato. Rilasciare il lock e ritornare un errore (ESRCH).